MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE, ALWAR

Name of faculty: Mr Jitgarun Kumar Gindal (Asst Professor)

Subject & Subject Code: Internet of Things (6CS4-05)

Semester: VI

Session: 2024 (Even Sem)

Branch: CSE

Unit No.: I

Date of Submission: 01/03/24

Lecture No.	Topics Covered	Page No.	Total Page
1	Sensor	1 - 17	17
2	Actuator	09	01
3	Humidity sensor	16 - 17	02
4	Temperature Sensor	23-25	03
5	Ultrasonic sensor	23 - 24	02
6	Arduino	25 - 39	15
7	Raspberry Pi	40 - 49	10
8	LiteOS	49 - 50	02
9	RiotOS	54 - 55	02
10	Contiki OS	56 - 59	04
11	Tiny OS	60	01

References:

1. IoT Fundamentals: Networking Technologies, Protocols and Use Cases for Internet of Things, David Hanes, Gonzalo Salgueiro, Patrick Grossetete, Rob Barton and Jerome Henry, Cisco Press, 2017
2. Internet of Things – A hands-on approach, Arshdeep Bahga, Vijay Madisetti, Universities Press, 2015

Faculty

HOD

Date Academics 1/3/24

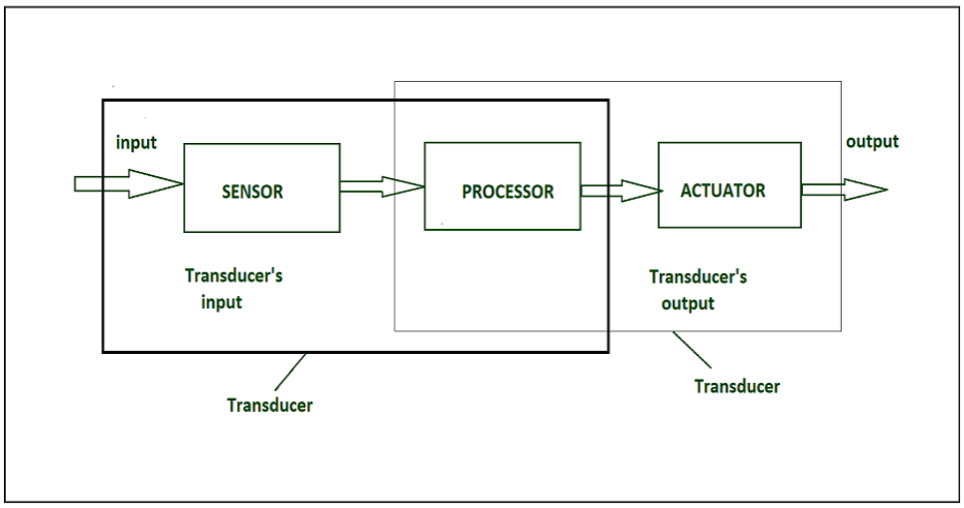
Unit-2

Sensors in Internet of Things(IoT):

Generally, sensors are used in the architecture of IOT devices.

Sensors are used for sensing things and devices etc.

A device that provides a usable output in response to a specified measurement. The sensor attains a physical parameter and converts it into a signal suitable for processing (e.g. electrical, mechanical, optical) the characteristics of any device or material to detect the presence of a particular physical quantity.
The output of the sensor is a signal which is converted to a human-readable form like changes in characteristics, changes in resistance, capacitance, impedance, etc.



IOT HARDWARE

Transducer :

- A transducer converts a signal from one physical structure to another.
- It converts one type of energy into another type.
- It might be used as actuator in various systems.

Sensors characteristics :

1. Static
2. Dynamic

1. Static characteristics:

It is about how the output of a sensor changes in response to an input change after steady state condition.

- **Accuracy:** Accuracy is the capability of measuring instruments to give a result close to the true value of the measured quantity. It measures errors. It is measured by absolute and relative errors. Express the correctness of the output compared to a higher prior system. Absolute error = Measured value – True value Relative error = Measured value/True value
- **Range:** Gives the highest and the lowest value of the physical quantity within which the sensor can actually sense. Beyond these values, there is no sense or no kind of response. e.g. RTD for measurement of temperature has a range of -200`c to 800`c.
- **Resolution:** Resolution is an important specification for selection of sensors. The higher the resolution, better the precision. When the accretion is zero to, it is called the threshold. Provide the smallest changes in the input that a sensor is able to sense.
- **Precision:** It is the capacity of a measuring instrument to give the same reading when repetitively measuring the same quantity under the same prescribed conditions.
  It implies agreement between successive readings, NOT closeness to the true value.
  It is related to the variance of a set of measurements.
  It is a necessary but not sufficient condition for accuracy.
- **Sensitivity:** Sensitivity indicates the ratio of incremental change in the response of the system with respect to incremental change in input parameters. It can be found from the slope of the output characteristics curve of a sensor. It is the smallest amount of difference in quantity that will change the instrument's reading.
- **Linearity:** The deviation of the sensor value curve from a particularly straight line. Linearity is determined by the calibration curve. The static calibration curve plots the output amplitude versus the input amplitude under static conditions.
  A curve's slope resemblance to a straight line describes linearity.
- **Drift:** The difference in the measurement of the sensor from a specific reading when kept at that value for a long period of time.
- **Repeatability:** The deviation between measurements in a sequence under the same conditions. The measurements have to be made under a short enough time duration so as not to allow significant long-term drift.

2. Dynamic Characteristics:

Properties of the systems:

- **Zero-order system:** The output shows a response to the input signal with no delay. It does not include energy-storing elements.
  Ex. potentiometer measure, linear and rotary displacements.
- **First-order system:** When the output approaches its final value gradually. Consists of an energy storage and dissipation element.
- **Second-order system:** Complex output response. The output response of the sensor oscillates before steady state.

Sensor Classification :

- Passive & Active
- Analog & digital
- Scalar & vector

1. **Passive Sensor:** – Can not independently sense the input. Ex- Accelerometer, soil moisture, water level and temperature sensors.
2. **Active Sensor:** – Independently sense the input. Example- Radar, sounder and laser altimeter sensors.
3. **Analog Sensor:** The response or output of the sensor is some continuous function of its input parameter. Ex- Temperature sensor, LDR, analog pressure sensor and analog hall effect.
4. **Digital Sensor:** – Response in binary nature. Design to overcome the disadvantages of analog sensors. Along with the analog sensor, it also comprises extra electronics for bit conversion. Example – Passive infrared (PIR) sensor and digital temperature sensor(DS1620).
5. **Scalar sensor:** – Detects the input parameter only based on its magnitude. The answer for the sensor is a function of magnitude of some input parameter. Not affected by the direction of input parameters. Example – temperature, gas, strain, color and smoke sensor.
6. **Vector Sensor:** – The response of the sensor depends on the magnitude of the direction and orientation of input parameter. Example – Accelerometer, gyroscope, magnetic field and motion detector sensors.

Types of sensors:

- **Electrical sensor :**

Electrical proximity sensors may be contact or non contact.

Simple contact sensors operate by making the sensor and the component complete an electrical circuit.

Non- contact electrical proximity sensors rely on the electrical principles of either induction for detecting metals or capacitance for detecting non metals as well.

- **Light sensor:**

Light sensor is also known as photo sensors and one of the important sensor.

Light dependent resistor or LDR is a simple light sensor available today.

The property of LDR is that its resistance is inversely proportional to the intensity of the ambient light i.e when the intensity of light increases, it's resistance decreases and vise versa.

- **Touch sensor:**

Detection of something like a touch of finger or a stylus is known as touch sensor.

It's name suggests that detection of something.

They are classified into two types:

1. Resistive type
2. Capacitive type

Today almost all modern touch sensors are of capacitive types.

Because they are more accurate and have better signal to noise ratio.

- **Range sensing:**

Range sensing concerns detecting how near or far a component is from the sensing position, although they can also be used as proximity sensors.

Distance or range sensors use non-contact analog techniques. Short range sensing, between a few millimetres and a few hundred millimetres is carried out using electrical capacitance, inductance and magnetic technique.

Longer range sensing is carried out using transmitted energy waves of various types eg radio waves, sound waves and lasers.

- **Mechanical sensor:**

Any suitable mechanical / electrical switch may be adopted but because a certain amount of force is required to operate a mechanical switch it is common to use micro-switches.

- **Pneumatic sensor:**

These proximity sensors operate by breaking or disturbing an air flow.

The pneumatic proximity sensor is an example of a contact type sensor. These cannot be used where light components may be blown away.

- **Optical sensor:**

In there simplest form, optical proximity sensors operate by breaking a light beam which falls onto a light sensitive device such as a photocell. These are examples of non contact sensors. Care must be exercised with the lighting environment of these sensors for example optical sensors can be blinded by flashes from arc welding processes, airborne dust and smoke clouds may impede light transmission etc.

- **Speed Sensor:**

Sensor used for detecting the speed of any object or vehicle which is in motion is known as speed sensor .For example – Wind Speed Sensors, Speedometer ,UDAR ,Ground Speed Radar .

- **Temperature Sensor:**

Devices which monitors and tracks the temperature and gives temperature's measurement as an electrical signal are termed as temperature sensors .These electrical signals will be in the form of voltage and is directly proportional to the temperature measurement .

- **PIR Sensor:**

PIR stands for passive infrared sensor and it is an electronic sensor that is used for the tracking and measurement of infrared (IR) light radiating from objects in its field of view and is also known as Pyroelectric sensor .It is mainly used for detecting human motion and movement detection .

- **Ultrasonic Sensor:**

The principle of ultrasonic sensor is similar to the working principle of SONAR or RADAR in which the interpretation of echoes from radio or sound waves to evaluate the attributes of a target by generating the high frequency sound waves

Sensors – II

Digital Sensors:

We now address sensors that do not require an external ADC, but report their measurement values directly to the microcontroller in digital from. Some sensorsalready have the ADC built in, and others do not need one. We start with the latter,of which the most prominent examples are buttons and switches.

Buttons and switches:

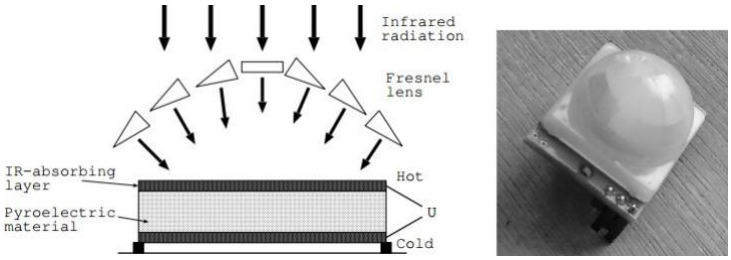
The simplest digital sensor is certainly a switch that is either closed or open, or a button where the open or close state is only activated temporarily. We use these terms interchangeably. Our task is to sense their state in a reliable way, and this is normally done with a pull-up resistor that is connected to the supply voltage in theway shown in Figure 2.31. In this way the sensing pin on the microcontroller can reliably detect the supply voltage. Only if the switch S1 is pressed does the voltage on the pin drop to zero or ground level. Only a small current, determined by the magnitude of the resistor, flows when the switch is closed. The actual value of the resistor is uncritical, but values around 10 to 30 k are usually reasonable. Without the pull-up resistor, the voltage potential on the pin is undefined when the switch isopen, and determined by stray capacitances in the system. So, the recommendationis to always use a pull-up resistor when sensing the state of a switch. Note that swapping the position of the resistor and the switch, the resistor functions as a pull-down resistor, and the sensing level is zero unless the switch is closed.

On/off devices:

A number of sensors provide a voltage level to inform the microcontroller about their state or change of state. They can be thought of as a switch with a built-in pull-up resistor and can be sensed in the same way. A problem can occur if the operating voltage level of the sensor and the microcontroller do not agree. Nowadays many sensors operate on levels of 2.5 to 3.3 V, and microcontrollers on levels from 2.5 to 5 V. Sensing higher external voltages such as those used in cars (12 V) or industrial control applications (24 or 48 V) requires some adjustment of the voltage level, to prevent damaging either the sensor or the microcontroller. There are level-changing chips available, such as the 74LVC245, but in many cases a simple voltage divider with two resistors is adequate. Yet, it only works if there is signal flowing from the high to the low-voltage side. In case a bidirectional signal flow is necessary, such as on the data line of the I2C bus, the solution shown in Figure 2.32, based on an n-type MOSFET transistor, is easy to implement. In the first case, when both logic signals are high, the MOSFET is not conducting, because the voltage difference between gate and source is close to zero. In the second case, if the 3.3 V logic is controlling and the signal is pulled low, the difference between gate and source is positive and the MOSFET conducts, such that even the 5 V logic level is pulled low. In the third case, when the 5 V logic is controlling and the 5 V logic signal is pulled low, the built-in diode (visible in the schematics) conducts, and causes the voltage of the source to drop to about 0.7 V. At this point the gate source voltage drop is sufficiently large to fully cause the MOSFET to conduct, which also pulls the 3.3 V logic level low.

But let us return to the sensors. A prominent device that reports its state througha changing voltage level is a PIR proximity sensor, shown on the right of Figure 2.33.It senses the change in the infrared radiation level, which announces the presence of living beings. The sensors are based on collecting the incident infrared radiation

with a Fresnel lens, which is the dome visible in Figure 2.33, on a pyroelectric sensor. The lens is often made of polyethylene, a material chosen for its low absorption of IR radiation. One side of the sensor is consequently warmed up and expands, which causes buckling of the piezo- or pyroelectric material, often a polymer film. Two effects contribute to the voltage between the upper and lower plates. First, the buckling strains the material and causes a piezoelectric voltage. Second, heat flows from the hot to the cold side and adds a pyroelectric voltage.



I2C devices:

A large number of sensors have some logic built in and support a high-level communication protocol. An example is the I2C protocol operating on the I2C bus. The physical connection to devices supporting I2C only needs four wires: Ground, supply voltage Vcc, clock SCL, and data SDA. The latter two require a pull-up resistor, which is often already included in the microcontroller that also serves as the I2C busmaster to orchestrate the communication. It configures the sensor, initiates a measurement, and then reads data from the sensor. Physically, the communication is based on a synchronous serial protocol, where the data line is sampled every time the clock line changes from a high level to a low level. The protocol is standardized andwe will not go into details, but mention that the I2C devices and also I2C sensors have a number of registers internal that can be written to in order to configure the sensor, or read from in order to retrieve sensor data. The communication is entirely

based on exchanging digital signals, and, as mentioned before, coordinated by the busmaster which normally is a microcontroller. Several devices can share the same SDA and SCL lines, because each device has its own address and responds only to those messages intended for it by specifying the device address.

SPI devices:

The SPI interface is a synchronous serial communication bus, similar to the I2C bus, but it can operate at much higher speed and is therefore often used for devices that require the continuous transfer of large amounts of data, such as displays or audio equipment. SPI communication requires one master on the bus, a role normally taken by a microcontroller. The sensors are typically slave devices. They need at least six wires to connect: ground and supply voltage, the clock CLK, one line to send information from the master to the slave (MOSI, for master-out slave-in), oneline for the reverse direction (MISO for master-in slave-out), and a chip-select line CS to identify the currently active slave. CLK, MISO, and MOSI lines can be shared among many slaves, but each slave requires its individual CS line.

RS-232 devices:

Several devices report their measurement values by sending them via the asynchronous RS- 232 protocol. Originally, the physical medium for the communication channel used a current loop, but nowadays most sensors operate on 3.3 or 5 V levels. The communication happens point-to-point between two partners who have agreed on a communication speed, which is often 9600 baud or 115200 baud. Three wires are required at the minimum, one for ground potential, one, labeled TX for transmitting from device A to device B, and another one, labeled RX, for transmitting in the reverse direction. To establish communication, cables connect the TX pin on one device to RX on the other and vice-versa. Naturally, ground pins need to be

connected as well.

One device that supports RS-232 communication is the LV-EZx distance sensor we already discussed earlier. It can be configured to send the distance measured asan ASCII string that can be read in a terminal program.

Sensors that query the global positioning system (GPS) use a small patch antenna on the sensor to pick up signals from a number of satellites placed in geostationary orbits, which broadcast their position and timing information with high precision. On-board electronics that normally comprise a microcontroller use triangulation in order to determine the position of the sensor with high accuracy and convert that information to an ASCII string containing the position in a standardized format, called NMEA. The string is written, typically once per second, to an RS-232 serial line, where it is straightforward to read and decode.

Actuators:

An actuator is a mechanical device that converts energy into motion. Actuators are used in a variety of applications, from controlling valves and switches in industrial settings to moving parts in robotics and aerospace engineering.

There are several types of actuators, including:

Electric Actuators: These use electricity to produce motion. They can be further classified into:
Linear actuators: produce motion in a straight line.
Rotary actuators: produce rotational motion.
Hydraulic Actuators: These use hydraulic fluid to produce motion. They are used in heavy-duty applications where high force is required, such as in construction equipment.
Pneumatic Actuators: These use compressed air to produce motion. They are often used in situations where electric or hydraulic actuators are not suitable, such as in explosive environments or in applications where electricity is not readily available.
Mechanical Actuators: These use mechanical means, such as a screw or lever, to produce motion. They are simple and cost-effective but may not be as precise or powerful as other types of actuators.
Piezoelectric Actuators: These use piezoelectric materials, which change shape when an electric field is applied, to produce motion. They are used in precision applications, such as in nanopositioning systems.

Actuators are an essential part of many systems and play a crucial role in controlling and manipulating physical processes.

IoT Sensors:

When it comes to the Internet of Things, or the network of smart devices, IoT sensors play a critical role. Essentially, these sensors are integrated into devices like smartphones, doorbells, thermostats and even cars and refrigerators to capture important data that can be shared in real time with users.

Increasingly, cities, municipalities and buildings are also using devices with IoT sensors to monitor, analyze and manage resources, weather, traffic and heating and cooling systems.

IOT SENSORS:

1. Accelerometers
2. Air quality sensors
3. Biomedical sensors
4. Cameras
5. Chemical sensors
6. Electric current sensors
7. Flow sensors
8. Gyroscopes
9. Humidity sensors
10. Level sensors
11. Motion sensors
12. Pressure sensors
13. Proximity sensors
14. Temperature sensors
15. Ultrasonic sensors

What Is an IoT sensor?

An IoT sensor is any sort of mechanism or tool, such as a camera or air quality monitor, integrated into a device. These sensors gather information — like water levels, air temperature and traffic patterns — related to the environments in which they're deployed and transmit it to the cloud via Wi-Fi, bluetooth, 5G or other mobile network.

Take smart thermostats, for instance, which are equipped with temperature sensors that measure fluctuations in a home's temperature. Because these thermostats are connected to the internet, users can log in via an app to monitor the temperature of their home and control the thermostat remotely.

What Are IoT Sensors Used For?

IoT sensors gather information so devices can be used remotely and data can be shared in real time. The data gathered by IoT sensors and sent to the cloud is analyzed by software that can make sense of the information and then sent to users. This data is used to track trends and gather insights about everything from efficiency and energy use in factories to athletic performance and a user's health.

Types of IoT Sensors:

These are some of the most popular IoT sensors found in smart devices like wearables, smartphones and other monitoring tools used by individuals, organizations and cities.

An overview of the differences between accelerometers and gyroscopes.

ACCELEROMETERS:

Accelerometers measure acceleration — basically the change in speed, direction and intensity of movement.

How is the data from these IoT sensors used? Accelerometers are used to collect data from connected devices such as wearables, medical alert devices, cameras and cars to track activity. Tablets and smartphones also rely on accelerometers to know when to rotate displays based on the device's physical orientation.



Air quality sensors gather data about what's in the air and transmit their findings to the cloud. |

AIR QUALITY SENSORS:

Air quality sensors measure levels of pollution, carbon dioxide and other particulates in the air.

How is the data from these IoT sensors used? Air quality sensors gather data about what's in the air and transmit their findings to the cloud. Cities typically use devices with air quality sensors so they can access real-time air quality conditions to issue warnings to residents if needed and track trends for study. Air quality sensors are being used in homes as well, detecting harmful particulates that could be indoors from appliances like gas stoves, which could warn residents to open a window or turn on an exhaust fan.



Wearable devices like smartwatches and arm bands can be equipped with biomedical sensors. |

BIOMEDICAL SENSORS:

Biomedical sensors measure a person's vital signs, like heart rate and oxygen level.

How is the data from these IoT sensors used? Wearable devices like smartwatches and arm bands can be equipped with biomedical sensors. These devices communicate the data they collect back to the wearer so they can monitor their own health, or to a caregiver or medical staff, who is monitoring a patient or loved one remotely.



Ring cameras notify a homeowner when someone is at their door, even if they aren't there themselves.

CAMERAS:

Cameras and image sensors capture visual data and images.

How is the data from these IoT sensors used? Everything from smart refrigerators to autonomous vehicles rely on cameras to detect objects and monitor their surroundings. Cameras are also critical to smart cities as well as home and business security — think Ring cameras notifying a homeowner when someone is at their door, even if they aren't there themselves. Cameras also play a role in factory and warehouse automation, helping perform tasks like product inspections.



Chemical sensors are used for environmental monitoring, like detecting chemical leaks in factories.

CHEMICAL SENSORS:

Chemical sensors detect chemical changes like radiation and pH levels in air or liquids.

How is the data from these IoT sensors used? Chemical sensors are used for environmental monitoring, like detecting chemical leaks in factories. They're also being integrated into wearable devices to monitor the chemical makeup of sweat, saliva and even tears for more timely treatment and therapeutic care.



Electric current sensors are often used to track energy consumption and remote and real-time monitoring of power systems.

ELECTRIC CURRENT SENSORS:

These sensors measure the amount of electricity running through a wire. They do so by monitoring the magnetic field the current generates along the wire.

How is the data from these IoT sensors used? Electric current sensors are often used to track energy consumption and for remote and real-time monitoring of power systems to manage energy needs in places like data centers.



Flow sensors are also a key component of smart metering devices, which monitor how water or other resources like natural gas are used and consumed.

FLOW SENSORS:

Flow sensors measure how quickly a liquid or gas flows past a certain point in a tube or pipe. A flow rate that's too fast or too slow could indicate a problem, like a leak.

How is the data from these IoT sensors used? Cities typically use flow sensors to help manage water systems and detect leaks. These sensors are also a key component of smart metering devices, which monitor how water or other resources like natural gas are used and consumed.

HUMIDITY SENSORS:

Humidity sensors monitor humidity, or the amount of water vapor, in the air.

How is the data from these IoT sensors used? These sensors are often used in smart building management and industrial settings to better manage comfort and energy use. They are typically paired with temperature sensors and other sensors like water sensors for remote monitoring of the environment.

Humidity sensors are essential components in the Internet of Things (IoT) applications where environmental conditions need to be monitored and controlled. They measure the amount of moisture or water vapor in the air, usually expressed as a percentage of the total amount of moisture the air could hold at a given temperature.
There are various types of humidity sensors available, with different operating principles and sensing technologies:

Capacitive Humidity Sensors: These sensors operate on the principle of capacitance change with humidity. As humidity increases, the dielectric constant of the material between the conductors increases, resulting in a change in capacitance. This change in capacitance is then converted into a humidity reading.

Resistive Humidity Sensors: These sensors measure humidity by monitoring the change in resistance of a moisture-sensitive layer (typically made of polymer or ceramics) when exposed to humidity. As the moisture content changes, so does the resistance, which is then translated into a humidity value.

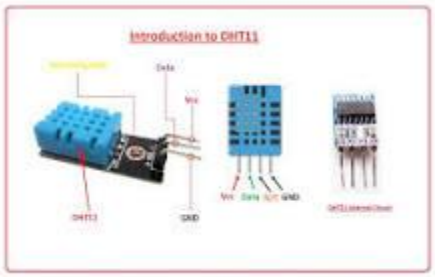
Thermal Humidity Sensors: Thermal sensors measure the rate of heat transfer between a heated surface and the ambient air. As humidity increases, the heat transfer rate changes, which is used to calculate the humidity.
Conductive Humidity Sensors: These sensors work by measuring the electrical conductivity of a material that changes with humidity. They use a set of electrodes and are sensitive to changes in the electrical properties of the surrounding environment.

Optical Humidity Sensors: These sensors use optical elements (such as light-emitting diodes and photodiodes) to detect changes in the refractive index of a moisture-sensitive material. The change in refractive index is then correlated to the humidity level.

Piezoelectric Humidity Sensors: These sensors utilize piezoelectric materials, which generate an electrical charge in response to mechanical stress, to measure humidity. The presence of moisture causes the piezoelectric material to expand or contract, generating a corresponding electrical signal.

Hygrometers: These are traditional instruments used to measure humidity and often consist of a hair or a fiber bundle that expands or contracts with humidity changes.

It's important to choose a humidity sensor that suits your application's needs in terms of accuracy, response time, durability, power consumption, and cost. Additionally, calibration and sensor drift are important factors to consider when using humidity sensors for long-term applications.







Level sensors monitor the level of liquids in a contained environment.

LEVEL SENSORS:

Level sensors monitor the level of liquids in a contained environment.

How is the data from these IoT sensors used? The data these sensors gather and report are critical when it comes to leak detection in industrial settings.



Safety and surveillance can detect the presence of an object or person from the heat it emits or by bouncing ultrasonic waves off the object.

MOTION SENSORS:

These sensors use infrared radiation or ultrasonic waves to detect motion.

How is the data from these IoT sensors used? Safety and surveillance is a common use case as motion sensors can detect the presence of an object or person from the heat it emits or by bouncing ultrasonic waves off the object.

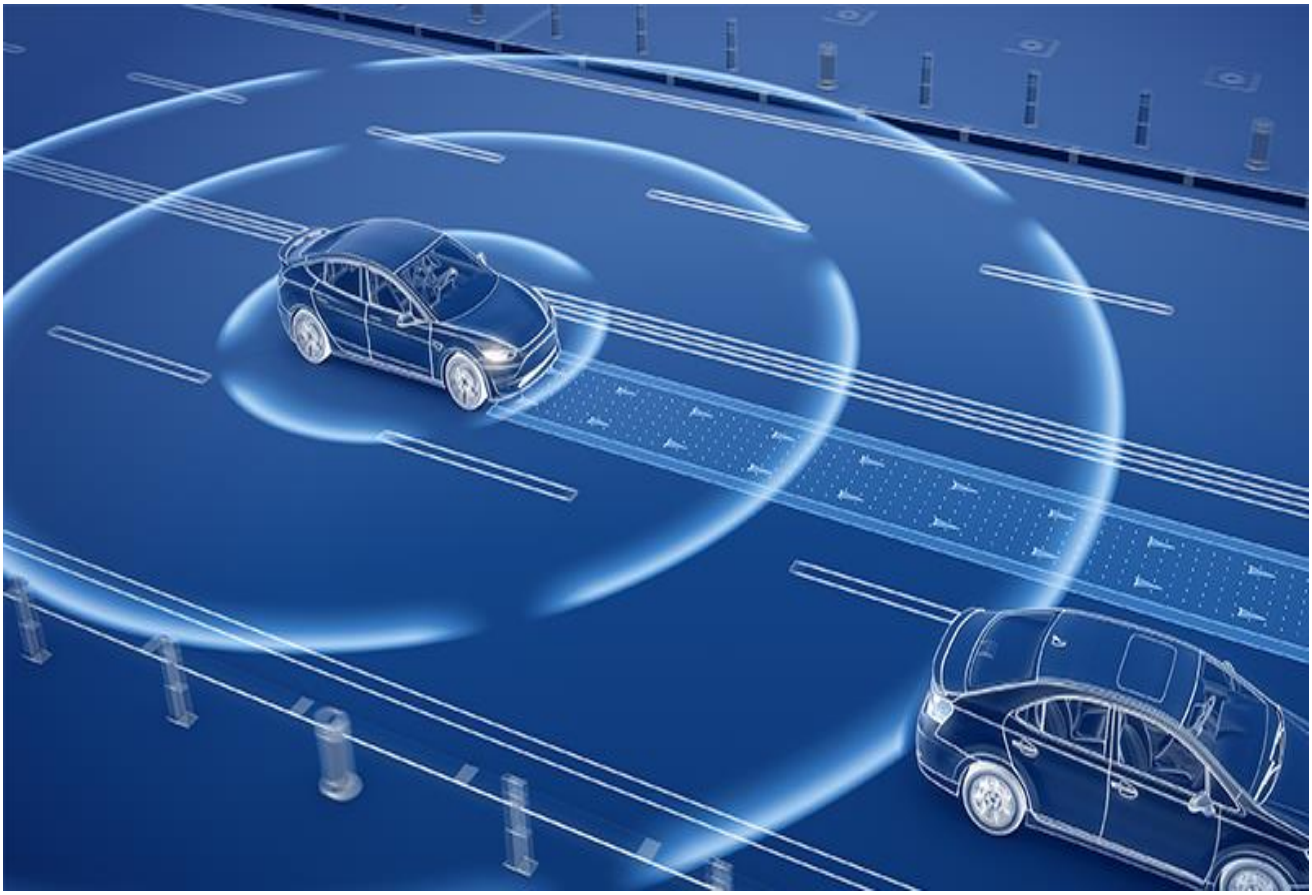


Pressure sensors, which are used to detect the rise and fall of pressure, are also used for leak detection.

PRESSURE SENSORS:

Pressure sensors measure the amount of pressure applied to a liquid or gas.

How is the data from these IoT sensors used? Pressure sensors, which are used to detect the rise and fall of pressure, are used for leak detection and used to maintain water and heating systems in real time.



Proximity sensors also use infrared radiation or ultrasonic waves to measure the distance between objects like cars.

PROXIMITY SENSORS:

These sensors bounce infrared radiation or ultrasonic waves off objects to detect their presence.

How is the data from these IoT sensors used? Proximity sensors, which are similar to motion sensors, also use infrared radiation or ultrasonic waves to measure the distance between objects, which is used in vehicles, smart cities for monitoring traffic, and industrial settings to track how parts, people and other objects move through a factory.



Temperature sensors are used to monitor air or water temperature and are common in smart buildings and smart thermostats.

TEMPERATURE SENSORS:

Temperature sensors detect changes in temperature, measuring the amount of heat there is in an environment.

How is the data from these IoT sensors used? Temperature sensors are used to monitor air or water temperature and are common in smart buildings and smart thermostats. Increasingly, food transport companies use these sensors to monitor any rise or fall in temperature of food storage areas. They're also used in agriculture to monitor the temperature of soil and can provide real-time updates to farmers.

Temperature sensors are crucial components of IoT systems, especially for applications like smart homes, agriculture, industrial monitoring, and weather tracking. These sensors help monitor and maintain optimal environmental conditions, enhance energy efficiency, and ensure equipment longevity.

Types of Temperature Sensors:

Digital Temperature Sensors:
Digital Temperature Sensors convert temperature to a digital signal, making them compatible with microcontrollers and other digital devices.
Thermistors are resistors whose resistance varies with temperature. There are two types: Negative Temperature Coefficient (NTC) and Positive Temperature Coefficient (PTC).
Digital Thermometers use integrated circuitry to measure temperature and often include additional features like LCD displays and alarm functions.
Dallas Semiconductor's DS18B20 is a popular digital temperature sensor with a single-wire interface, allowing multiple sensors to connect to a single microcontroller pin.

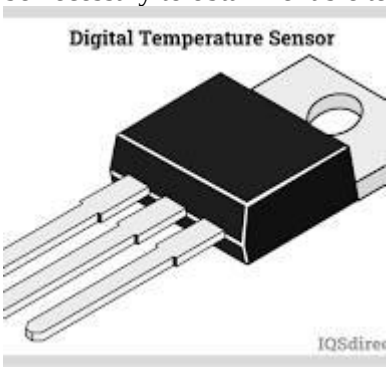
Analog Temperature Sensors:
These sensors produce a continuous voltage or current signal corresponding to the temperature.
Thermocouples generate voltage based on the difference in temperature between two junctions made of different metals.
Resistance Temperature Detectors (RTDs) use the principle that the resistance of a metal increases with temperature.
Thermistors also belong to this category, but they can be both analog and digital.

Integrated Circuit (IC) Temperature Sensors:

These sensors are embedded within microcontrollers and provide temperature readings along with other functionalities.
Microchip Technology's MCP9808 is an example of an IC temperature sensor with high accuracy and a digital interface.

Infrared (IR) Temperature Sensors:
These sensors measure temperature remotely using infrared radiation emitted by objects.
Non-contact IR thermometers are commonly used in industrial applications where direct contact with the object is not feasible.

For IoT projects, it's essential to choose a temperature sensor that suits your specific requirements in terms of accuracy, range, response time, cost, and power consumption. Calibration, noise reduction, and data smoothing techniques may also be necessary to obtain reliable temperature readings in real-world applications.





Ultrasonic Sensor:

Ultrasonic sensors are devices that use ultrasonic waves to measure the distance between the sensor and an object. They are commonly used in robotics, automotive parking sensors, industrial automation, and security systems.

Here's how an ultrasonic sensor typically works:

Transmitter: The sensor emits ultrasonic waves, usually between 20 kHz to 200 kHz, which are inaudible to humans.
Reflection: When the ultrasonic waves encounter an object, they are reflected back to the sensor.
Receiver: The sensor detects the reflected waves.
Calculation: The time taken for the ultrasonic waves to travel to the object and back is measured. The distance between the sensor and the object is calculated using the formula:

Distance=Speed×Time2Distance=2Speed×Time

where the speed is the speed of sound in air (approximately 343 meters per second at 20°C).
Output: The calculated distance is typically output in terms of a voltage, digital signal, or through a communication protocol such as I2C, SPI, or UART.

There are different types of ultrasonic sensors:

2-wire Ultrasonic Sensors: These sensors have two wires for power and ground, and they output an analog voltage or current proportional to the distance.

3-wire Ultrasonic Sensors: These sensors have three wires for power, ground, and an output signal that represents the measured distance.

4-wire Ultrasonic Sensors: These sensors have four wires, including power, ground, and separate trigger and echo signals, making them more versatile.

Ultrasonic Range Finders: These sensors are specifically designed to measure long distances with higher accuracy. They often include features such as multiple transducers for improved sensitivity.

Ultrasonic Proximity Sensors: These sensors are designed to detect the presence or absence of an object within a specific range.

Multi-Sensor Arrays: Some ultrasonic sensors include multiple transducers, allowing for more advanced sensing capabilities, such as 360-degree coverage or object tracking.

Ultrasonic sensors have various applications in robotics, smart devices, industrial automation, automotive systems, and more. They are generally reliable and easy to use but may be affected by factors such as temperature, humidity, and the reflective properties of objects. Calibrating the sensor and accounting for environmental conditions can help ensure accurate distance measurements.



Arduino I:

The original Arduinos are based on Atmel microcontrollers and we mostly discuss the Arduino UNO. Support for a second family of controllers, based on the ESP8266 microcontroller, was recently integrated into the Arduino development environment. These controllers can be programmed in much the same way as UNOs, but have native wireless support built in. But let?s start with the UNOs.

Arduino UNO:

An Arduino UNO is shown in Figure 4.1, where the main component is the ATmega328p microcontroller from Atmel (now MicrochipTM), which is the large chip with 28 legs in the image. It is a controller with 8-bit-wide registers, and operates at a clock frequency of 16 MHz. It has 32 kB RAM memory and 1 kB non-volatile EEPROM memory, which can be used to store persistent data that need to survive tuning off and on the supply voltage. There are three timers on board, which are basically counters that count clock cycles and are programmable to perform some action, oncea counter reaches some value. The UNO interacts with its environment through 13 digital inputoutput (IO) pins, of which most can be configured to be either inputor output, and have software-configurable pull-up resistors. Several of the pins areconfigurable to support I2C, SPI, and RS-232 communication. Moreover, thereare six analog input pins. They measure voltages of up to the supply voltage of5 V. An alternative internal reference voltage source provides a 1.1 V reference.All digital and analog pins are routed to pin headers that are visible on bothsides of the Arduino printed circuit board (PCB) in Figure 4.1. Furthermore, the built-in hardware RS-232 port is connected to an RS-232-to-USB converter that

allows communication and programming from a host computer. There is no WiFi, Bluetooth, or Ethernet support on the UNO board, but extension boards, so-called shields, are available for mounting directly onto the pin headers.

One can describe the Arduino UNO as having the intelligence of a washing machine. It keeps time with the timers, it can sense voltages from, for example, temperature sensors, and it can turn motors or pumps on or off, depending on whether some condition is met. In this way it can also provide the glue logic tointerface sensors (analog, I2C, SPI, other) to the host computer, and that is the mode in which we will use the UNO later on.

Arduino IDE:

Once the installation completes, we start the IDE by clicking the icon, or start it from the command line by typing arduino followed by Enter, which should open theIDE and show us a “indow similar to the one shown in Figure 4.3. If not, create a ”New” sketch, which is what Arduino programs are called, by selecting “New” from the “File” menu.

Here we already see the general structure of Arduino programs (or synonymously “sketches”). There is a setup() function, which is executed once, immediately after power is turned on. In this routine all initialization housekeeping is done, suchas defining a pin to be output or input, and configuring the serial line. Once the setup() function completes, the loop() function is called repetitively, such that once it completes, it is called again and so forth, until power is turned off. Note that the programming language supported by the Arduino IDE is very similar to the C language. There are, however, a number of special extensions to provide access to the specific hardware, such as the ADC

Now we connect the Arduino UNO to any USB port on the host computer where the Arduino IDE is running, and select Arduino UNO from the Tools!Board menu.

This step tells the IDE for which processor the compiler will generate code, as well as some hardwarespecific definitions such as the names of IO pins that we can use when programming. At this point the IDE "knows" what the hardware is, but we still need to tell the IDE to which USB port the UNO is connected. This we do inthe Tools!Port menu, where normally the serial port to which the UNO is connected automaticlly appears and can be selected. On Linux this often is /dev/ttyUSB0 or /dev/ttyACM0. On a Windows computer it is COMx where x is some number.

A Sample Program

```
void setup() {
pinMode(LED_BUILTIN,
OUTPUT);
}
void loop() {
digitalWrite(LED_BUILTIN, HIGH); // turn the LED
ondelay(1000); // wait for a second
digitalWrite(LED_BUILTIN, LOW); // turn the LED
offdelay(1000); // wait for a second
}
```

In the setup() function, we call the pin Mode(Pin,What) function, which declares that the pin called LED BUILTIN (which is pin 13 on a UNO) will be used as outputpin. That is all the initializing we do in the setup() function. In the loop() we call digital write(Pin,State), which causes the controller to put 5 V onto the specified pin if the requested state is HIGH and 0 V if the requested state is LOW. The latter happens two lines later. In between the changes to the output pin, we tell the controller to wait for a specified number of milliseconds in the delay(time in ms) function. So, all the loop function does is turn on the LED on pin 13, wait for 1000 ms or 1 s, turn it off, wait again for 1 s, and then start all over again. Note that all

commands are terminated by a semicolon, as is customary in the C-language.

Arduino II:

Interfacing Sensors:

Button:

In Figure 4.5, we show how to connect a button between ground and pin 2 of the Arduino, such that pressing the button will cause pin 2 to read 0 V or LOW. The following listing shows a sketch that causes the LED on pin 13 to light up, if the button is pressed.

```
// button_press, V. Ziemann, 161013
void setup() {
pinMode(2,INPUT_PULLUP); // button, default=HIGH
pinMode(13,OUTPUT); // LED
}
void loop() {
if
(digitalRead(2)==LOW) {
digitalWrite(13,HIGH);
} else {
digitalWrite(13,LOW);
}
delay(10);
}
```

The program follows the normal scheme of initializing the hardware in the setup() function. We first declare that pin 2 is an input pin, and we enable the internalpull-up resistor that causes a well-defined “high” state on the pin if no button is pressed. Pin 13 with the LED is declared as output, so we can turn it on and off from within

the sketch. In the loop() function, we check the state of the buttonby calling the digitalRead(Pin) function, and test whether pin 2 is LOW. If this is the case (note the double equal sign == in the comparison), the LED on pin 13 isturned on by calling digitalWrite(13,HIGH). If pin 2 is found not to be LOW, the LED on pin 13 is tuned off with digitalWrite(13,LOW);. Note the braces f and g and their use to define blocks of code in the if (..) f g else f g construction. After the if statement, a short delay() ensures that mechanical button bounces are ignored and that the processor has a little time for its internal affairs. This is not absolutely necessary, but good style.

Analog Input:

We place them in a voltage divider and then read the voltage change on the central tap, as shown on the left in Figure 2.1. Others, such as the LM35 temperature sensor, directly produce a voltage and in this section we show how to interface these sensors to the Arduino. First, we want to measure a voltage from one of the analog input pins; here we use A0. The left of Figure 4.6 shows how to connect a potentiometer asa variable voltage divider to the Arduino. One end of the potentiometer is connected to ground, the other end to the supply voltage, and the wiper with the variable tap of the potentiometer is connected to pin A0. Turning the axis of the potentiometer causes the voltage on the center tap to vary between ground and the 5 V supply voltage.

```
// Analog and serial communication, V. Ziemann, 161130
int inp,val;
void setup() {
Serial.begin(9600); // baud rate
}
```

```
void loop() {
if (Serial.available()) {
inp=Serial.read(); // read character from serial
val=analogRead(0); // read analog
Serial.print("Value is "); // and report back
Serial.println(val);
}
delay(50); // wait 50 ms
}
```

In the setup() function, we declare that we want to use the built-in serial (RS- 232) hardware port with a speed of 9600 baud, which is about 1000 characters persecond. The receive (RX) and transmit (TX) lines of the serial port are connected topin 0 and pin 1, but also routed to the RS232-USB converter on the UNO board thatis connected to the host computer. In the loop() function, we first check whether some communication from the host computer has arrived, and if that is the case, we read the character with the Serial.read(); command, but do nothing further withit. In the next line we read the analog pin A0 using the built-in ADC, and storethe value in the integer variable val. The value returned from the 10-bit ADC is a number between 0 and 1023 (= 210 ? 1) and not the voltage. Later we will show how to convert this value to a voltage. The result of the measurement is then sent via the serial line by Serial.print() and Serial.println() commands. The difference between the two is that the former does not send a carriage return character at theend of the message, and the latter does. After the if() f g statement at the end of the loop, again a small delay is added.

I2C and SPI:

I2C:

The device has four pins—check the datasheet to find out which pin does what—for ground, supply voltage, and the I2C lines SDA as SCL. The latter two are connected to pin A4 and A5 on the UNO because the I2C data and clock lines are routed to the same output pins as the analog pins A4 and A5. These pins cannot be used for analog measurements, in case they are used for I2C connectivity.

```
// Read MLX90614 IRthermometer, V. Ziemann,
170717#include <Wire.h>
const int MLX90614=0x5A; // I2C address
float getTemperature(uint8_t addr) { //. ..... getTemperature
uint16_t val;
uint8_t crc;
Wire.beginTransmission(MLX90614);
Wire.write(addr); // address
Wire.endTransmission(false);
Wire.requestFrom(MLX90614,3);
val = Wire.read();
val |= (Wire.read()<<8);
crc=Wire.read(); // not used
return -273.15+0.02*(float)val;
}
void setup() { //..... setup
Serial.begin(9600);
Wire.begin();
}
```

```
void loop() { //..... loop
float Ta=getTemperature(0x06); // address for ambient temp
float To=getTemperature(0x07); // address for object1 temp
Serial.print(Ta); Serial.print("\t"); Serial.println(To);
delay(1000);
}
```

First we have to include support for the I2C functionality by including the Wire.h header file, which also causes the compiler and linker to include the corresponding libraries. After we define the I2C address 0x5A of the sensor, which we find in the datasheet, we encapsulate the I2C communication in a separate function called getTemperature(). It receives the register address of the I2C device as input parameter and returns the associate temperature, properly scaled to degrees Celsius. The device also returns a byte that allows us to determine transmission errors, but we do not use that feature in this simple example. In the setup() function we only initialize Serial line and I2C communication. In the loop() function we use the getTemperature() function to retrieve the ambient temperature from address 0x06 and the object temperature from address 0x07, as described in the datasheet, and print both temperatures formatted with a tabulator to the Serial line. The entire process is then repeated after waiting 1000 ms.

SPI

Instead of bit banging the pins, we can also utilize the SPI library that comes with the Arduino IDE. The sketch, based on the ¡SPI.h¿ library, but otherwise equivalent to the previous one, is the following.

```
//.MCP3304,.V..Ziemann,.170726
#include.<SPI.h>
```

```
#define.CS.15.//.D8
int.mcp3304_read_adc(int.channel).{.//...0.to.3. .....read_adc
int.adcvalue=0,.b1=0,.hi=0,.lo=0,.sign=0;;
digitalWrite.(CS,.LOW);
byte.commandbits.=.B00001000;.//.Startbit+(diff=0)
commandbits.|=.channel.&.0x03;
SPI.transfer(commandbits);
b1.=.SPI.transfer(0x00);.//.always.D0=0
sign.=.b1.&.B00010000;
hi.=.b1.&.B00001111;
lo.=.SPI.transfer(0x00);.//.input.is.don?t.care
digitalWrite(CS,.HIGH);
adcvalue.=.(hi.<<.8).+.lo;
if.(sign).{adcvalue.=.adcvalue-4096;}
return.adcvalue;
}
void.setup().{//.....setup
pinMode(CS,OUTPUT);
SPI.begin();
SPI.setFrequency(2100000);
SPI.setBitOrder(MSBFIRST);
SPI.setDataMode(SPI_MODE0);
Serial.begin(115200);
while.(!Serial).{yield();}
}
void.loop().{.//.....loop
int.val=mcp3304_read_adc(0);
```

```
Serial.print("CH0.=.");.Serial.println(val);
delay(1000);
}
```

Here we first include the ¡SPI.h¿ functionality and define the CS pin before defining the mcp3304 read adc function to read a channel from the ADC. In this function we first declare a number of variables and pull the CS pin LOW in order to start the transaction. Then we build the commandbits; this time they are aligned such that the start bit is bit 3 and the channel information is stored in bits 1 and 0,and we use the SPI.transfer function to send it to the ADC. Note that the first bit recognized by the ADC is the first non-zero bit, which is the start bit 3. We then send 0x00 in a second call to SPI.transfer and receive

the readings from the MISO pin in b1. Since we have a number of idle clock toggles, the sign bit is bit 4 and the next four bits are the four most significant bits of the ADC reading, which we store in the variable hi. The next call to SPI.transfer() returns the subsequent bits, which are the lower eight bits from the MISO pin, and we store them in the variable lo. After pulling the CS pin HIGH to conclude the transaction, we build the ADC word adcvalue and add the sign information. Finally we return the reading to the calling program. In the setup() function we initialize the SPI functionality by calling the SPI.begin() function, setting the clock frequency, byte order, and MODE, and then initialize the serial line. The loop() function is a straight copy of the previous example, where we only read channel 0 and print the value to the serial line.

Interfacing Actuators:

Switching devices:

Dynamically adjusting the brightness of an LED is achieved by pulse-width modulation, and that feature is available on pins D3, D5, D6, D9, and D11 on the Arduino Uno. We use the analogWrite(pin,value) function that takes the pin number and a value between 0 and 255 (0 to 1023 on the ESP8266) to adjust the duty cycle of the pulse-width modulated signal from completely off to completely on. The switching frequency is between 500 and 1000 Hz, and in Figure 4.12, we show an oscilloscope trace of the result of analogWrite() to pin D9 of a UNO with values 88 and 220, respectively. The frequency of the signal is about 490 Hz, and we see the length of the signal differing from about 1/3 of the time at 5 V on the left to 5 V almost all the time on the right.

In order to be able to manipulate actuators in the same way as the sensors, weuse the same query-response protocol to communicate the required actions to the microcontroller. We want to use the convention that DWx 0 turns digital pin numberx off and DWx 1 turns it on, and that sending AWx nnn sets pulse-width modulationon pin number x to value nnn. The code that achieves this is the following:

```
// Switching_and_pwm, V. Ziemann, 170614
char line[30];
void setup() {
pinMode(2,OUTPUT);
Serial.begin (9600);
while (!Serial) {;}
}
void loop() {
if (Serial.available()) {
```

```
Serial.readStringUntil(?\n?).toCharArray(line,30);
if (strstr(line,"DW2 ")==line) {
int val=(int)atof(&line[4]);
digitalWrite(2,val);
} else if (strstr(line,"AW9 ")==line)
{int val=(int)atof(&line[4]);
analogWrite(9,val);
} else {
Serial.println("unknown");
}
}
}
```

DC motors

If we only want to control the speed of a very small motor, we can replace the LED and the 220 ? resistor in the previous example by the motor. We also need to add flyback diodes to prevent the back-emf from damaging the transistor. For slightly larger motors, we need to use a transistor with a higher power rating, such as a TIP-120 Darlington power transistor. The TIP-120 has a flyback diode from emitter to collector already built in. Note that the performance of larger transistors normally degrades at higher frequency. Inspection of the datasheet, however, shows that this only affects frequencies well above 10 kHz.

In Figure 4.14 we show the setup with a UNO controlling a small motor. The terminals of the transistor are base, collector, and emitter, from left to right, andthe motor is connected between the collector and the supply voltage. The emitter is directly connected to ground, and we include an external flyback diode (horizontally mounted) between the motor leads, with the cathode pointing towards the right. The

vertically mounted diode illustrates the connection of the built-in diode. We control the speed of the motor by pulse width modulating the base of the transistor that is connected to pin D9 on the UNO via a 1 k? resistor.

```
// H bridge DC motor Controller, V. Ziemann,
170614char line[30];
void setup() {
pinMode(2,OUTPUT);
pinMode(3,OUTPUT);
Serial.begin (9600);
while (!Serial) {;}
}
void loop() {
if (Serial.available()) {
Serial.readStringUntil(?\n?).toCharArray(line,30);
if (strstr(line,"FW ")==line) {
digitalWrite(2,LOW);
digitalWrite(3,LOW);
digitalWrite(3,HIGH);
float val=atof(&line[3]);
analogWrite(9,(int)val);
} else if (strstr(line,"BW ")==line)
{digitalWrite(2,LOW);
digitalWrite(3,LOW);
digitalWrite(2,HIGH);
float val=atof(&line[3]);
analogWrite(9,(int)val);
} else { // STOP in all other cases
```

```
digitalWrite(2,LOW);
digitalWrite(3,LOW);
analogWrite(9,(int)0);
}
}
}
```

Arduino is an open-source electronics platform based on easy-to-use hardware and software. It consists of a microcontroller board, an Integrated Development Environment (IDE) for writing code, and a community of users who contribute and share projects. It was developed to enable artists, designers, hobbyists, and anyone else interested in creating interactive projects to quickly and easily prototype their ideas.

Key components of the Arduino ecosystem include:

Microcontroller Boards: Arduino boards are equipped with a microcontroller, which is essentially a small computer that can be programmed to perform specific tasks.

Some common types of Arduino boards include:

Arduino Uno: This is one of the most popular and widely used Arduino boards. It features an Atmega328 microcontroller and has a wide range of I/O pins.

Arduino Mega: This board is similar to the Uno but with more I/O pins and memory.

Arduino Nano: This is a smaller version of the Uno, designed for more compact projects.

Arduino Due: This board is based on a more powerful microcontroller and is suitable for more complex projects.

Shields: Arduino shields are expansion boards that can be stacked on top of an Arduino board to add functionality.

Examples include motor control shields, Ethernet shields for networking, and LCD screens.

Software: The Arduino IDE is a software application used to write and upload code to Arduino boards. It's based on the Processing programming language and provides a simplified environment for programming microcontrollers.

Programming Language: The Arduino programming language is based on Wiring, a simplified version of C and C++. It provides functions for controlling the hardware components of the Arduino board, such as digital and analog input and output, serial communication, and more.

Libraries: Arduino libraries are pre-written code that can be easily incorporated into your projects. There are libraries available for various sensors, communication protocols, and other components, making it easier to interface with external

hardware.

Community: Arduino has a large and active community of users who share projects, tutorials, and code. This community support is invaluable for beginners and experienced users alike.

Arduino boards can be used for a wide range of projects, including home automation, robotics, wearable technology, environmental monitoring, and more.

The open-source nature of Arduino means that it can be easily customized and adapted to fit your specific needs. Whether you're a beginner looking to learn about electronics and programming or an experienced maker looking to prototype a new project, Arduino provides an accessible and powerful platform for building interactive projects.



Raspberry Pi I:

The Raspberry Pi7 is a series of single-board computers created in the United Kingdom by the Raspberry Pi Foundation. The original aim of the founders was to encourage the teaching of basic computer science in schools and developing countries. Step by step, their boards significantly changed the way manufacturers and developers thought about and created new projects in many application scenarios.For example, the original model became very popular and started spreading outside of its initial target market for uses such as robotics.

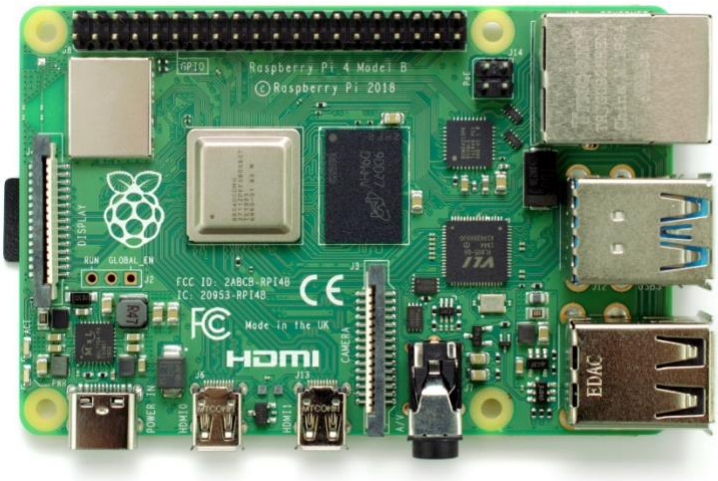
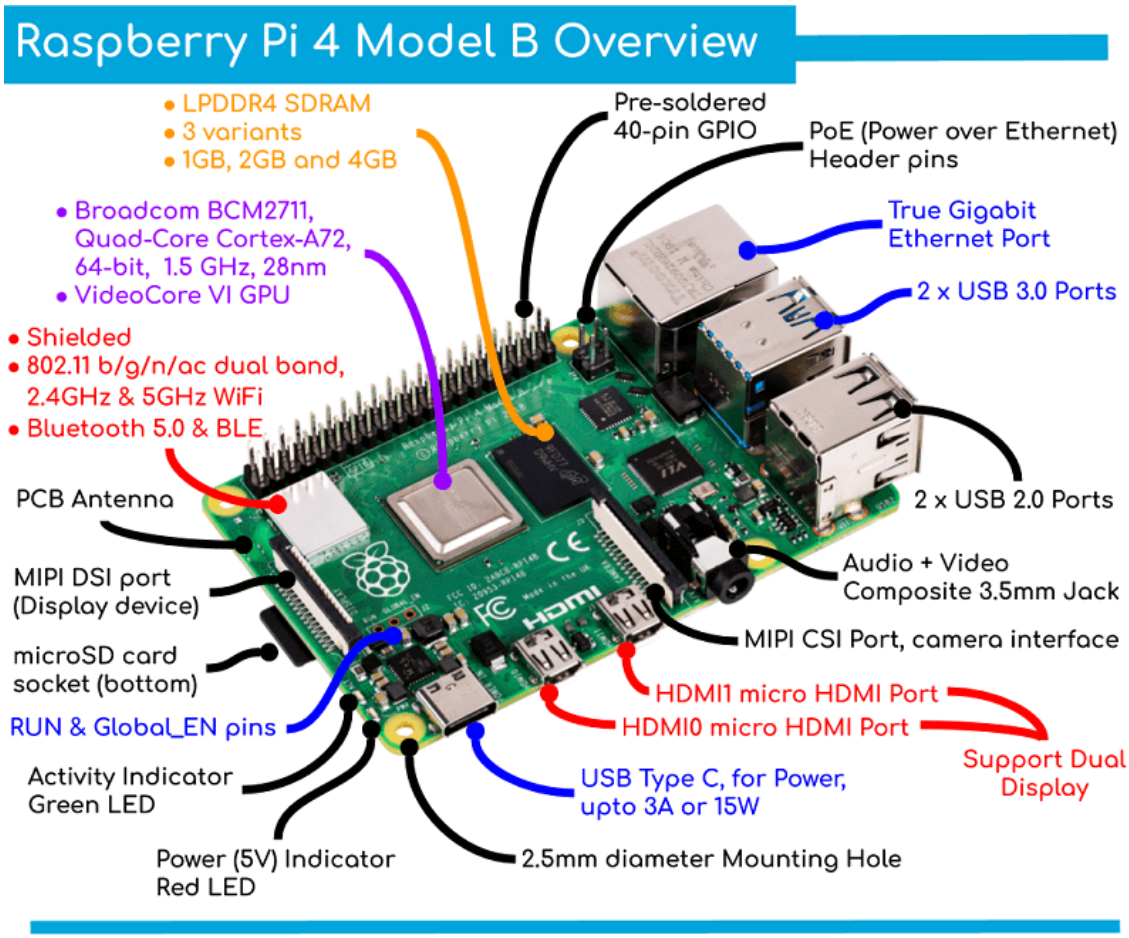


Figure 7.7 shows three Raspberry Pi boards with their available components and hardware profiles. The Broadcom BCM2835 SoC was used on the first generation and was inspired by the chip used in first-generation smartphones (its CPU is an older ARMv6 architecture). It includes a 700 MHz ARM1176JZF-S processor, VideoCore IV GPU, and RAM. It has a level 1 (L1) cache of 16 kB and a level2 (L2) cache of 128 kB. The level 2 cache is used primarily by the GPU. The SoC is stacked underneath the RAM chip, so only its edge is visible.

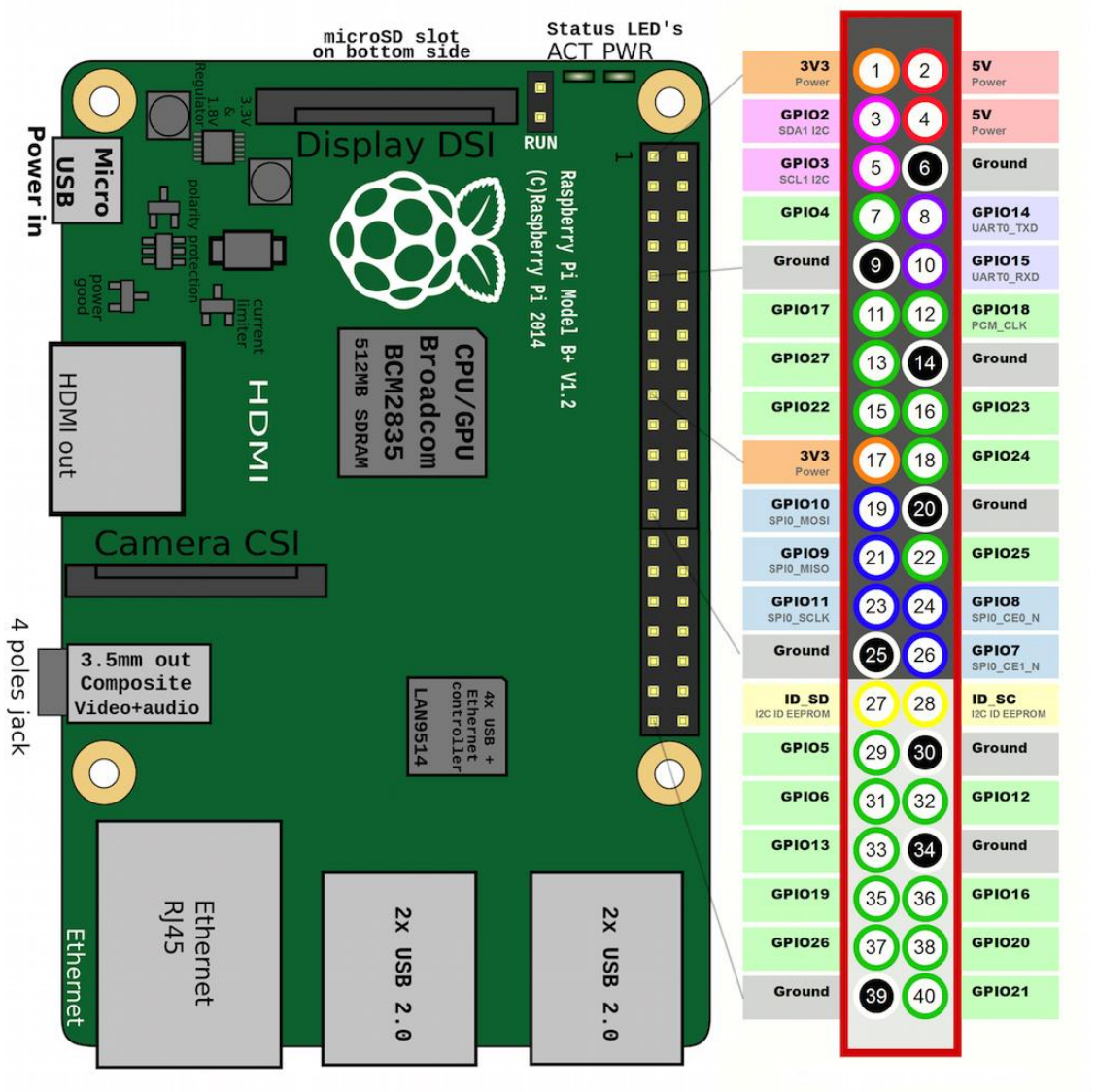
The Raspberry Pi 2 uses a Broadcom BCM2836 SoC with a 900 MHz 32-bit quad-core ARM Cortex-A7 processor (as do many current smartphones), with 256 kB shared L2 cache. The Raspberry Pi 3 uses a Broadcom BCM2837 SoC with a 1.2 GHz 64-bit quad-core ARM Cortex-A53 processor, with 512 kB shared L2 cache.



The Model A, A+ and Pi Zero are shipped without Ethernet modules and are commonly connected to a network using external adapters for Ethernet or Wi-Fi. Models B and B+ have the Ethernet port that is provided by a built-in USB Ethernet adapter using the SMSC LAN9514 chip. The Raspberry Pi 3 and Pi Zero W (wireless) provide a 2.4 GHz Wi-Fi 802.11n (150 Mbit/s) and Bluetooth 4.1 (24 Mbit/s) connectivity module based on a Broadcom BCM43438 chip. The Raspberry

Pi 3 is also equipped with a 10/100 Ethernet port.

The Raspberry Pi can be also used with USB storage, USB-to-MIDI converters, and virtually any other device/component with USB capabilities. Other external devices, sensors/actuators and peripherals can be attached through a set of pins and connectors available on the board's surface.



The Raspberry Pi board family can run multiple operating systems, such as Raspbian, Fedora, Ubuntu MATE, Kali Linux, Ubuntu Core, Windows 10 IoT Core, RISC OS, Slackware, Debian, Arch Linux ARM, and Android Things.

This combination of high-profile hardware, software and operating systems makes these boards to represent powerful and complex nodes in heterogenous IoT applications. They can efficiently work as IoT Hubs, gateways and data collectors using heterogenous protocols and running multiple services at the same time. All the Raspberry Piboards can be classified as Class 2 constrained devices.

But beyond the CPU, the Raspberry Pi sports a video processor that can display videos at full-HD resolution (1920×1080) via the built-in HDMI-connector. Audio output is available either via the HDMI connector or via a 3.5-mm headphone connector. Moreover, there are four USB-2 ports on board to connect peripheral components, such as USB sticks, keyboard, mice, or web cameras. Communication with the outside world is feasible via a built-in wired Ethernet port, and since version 3, the Raspi has had built-in Bluetooth (V4.1) and WiFi (802.11n).

The Raspis are very attractive due to their built-in low-level peripherals. There are 17 general-purpose inputoutput (GPIO) pins exposed on the board, some of which support I2C, SPI, and UART (RS-232-like) communication. Moreover, a specific audio bus (I2S) is available, as well as a high-speed CSI interface to connectthe tailor-made Raspberry Pi camera, and a DSI interface to connect LCD panels.

Raspberry Pi II:

Application:

RASPI AS A ROUTER:

On the Raspi we need to install the following packages

```
sudo apt-get update
sudo apt-get install hostapd dnsmasq
```

where we first update the repositories, and then download and install the hostapd and dnsmasq package. The former program is responsible for spanning the private wireless network, and turns the Raspi into a WLAN access point. The latter provides IP numbers on the private network via the DHCP protocol, and translates web site names to IP numbers. Next we need to re-configure our system. In the default configuration there is a background process, a daemon called dhcpd, that tries to obtain IP numbers for every network interface on the Raspi, including the wireless interface called wlan0. On the other hand, in order to operate the Raspi as an access point, we need to configure the interface ourselves and therefore need to remove the interface wlan0 from the control of the daemon by adding the line

```
denyinterfaces wlan0
```

at the end of the file /etc/dhcpcd.conf with any text editor, such as nano or emacs. Now we are ready to configure the interface wlan0 manually by rewriting the relevant section in the file /etc/network/interfaces to look like this:

```
auto wlan0
iface wlan0 inet static
address 192.168.20.1
```

```
netmask 255.255.255.0
up /sbin/iptables -A POSTROUTING –t nat -o eth0 –j
MASQUERADE down /sbin/iptables -D POSTROUTING –t nat -o
eth0 –j MASQUERADE
```

which defines the IP number of the wlan0 interface to be 192.168.20.1, and configures the type of network (class C) in the following line with the netmask command. The /sbin/iptables command enables network address translation between the wireless network on wlan0 and the wired network on interface eth0. In order to allow network packages to pass between the interfaces, which is disabled by default as a security measure, we need to enable forwarding by removing the hash from the line

```
net.ipv4.ip_forward=1
```

in the file /etc/sysctl.conf. We omit this step if all sensor nodes should only communicate with the Raspi, but no other computer on the local network should be able to surf the outside Internet beyond the Raspi, which might not be needed and poses a potential security risk. This last step completes the basic network setup, and we turn to the configuration of the access point software.

The configuration file for the hostapd daemon is /etc/hostapd/hostapd.conf and contains the following lines:

```
# /etc/hostapd/hostapd.conf
interface=wlan0
driver=nl80211
ssid=messnetz
channel=5
macaddr_acl=0
auth_algs=1
ignore_broadcast_ssid=0
wpa=2
```

wpa_passphrase=zxcvZXC
Vwpa_key_mgmt=WPA-
PSK wpa_pairwise=TKIP
rsn_pairwise=CCMP
hw_mode=g

The file must be made readable for the owner only by executing sudo chmod 600 /etc/hostapd/hostapd.conf. It contains the definition of the network name messnetz and the details of the encryption, such as the passphrase. The latter we should adapt to your own secret phrase, to prevent others from misusing your wireless network. We need to tell the system where to find the configuration file by entering the line

DAEMON_CONF=/etc/hostapd/hostapd.conf

near the top of the file /etc/init.d/hostapd and then instruct the system to startthe hostapd daemon at boot time with the command.

Communicating with Arduino:

From Commandline:

At this point we have established the Raspi as a development system for Arduino software, and ensure it communicates via the USB-based serial line with the Arduino IDE. But normally we want to access the Arduino from self-written programs rather than always using the Arduino IDE. The first option is to use the screen program or any other terminal program to connect to the UNO. After closing the Arduino IDE, we therefore start the following program on the command line on the Raspi:

screen /dev/ttyACM0 9600

just as we did earlier when the UNO was connected to the Desktop computer. Bluetooth communication works exactly the same way as on the desktop computer.

Python:

We use the following Python script to query the UNO with the query-response sketch for a single value from analog pin 0.

```
# query_arduino.py
import serial, time
query="A0?\n"
ser=serial.Serial("/dev/ttyACM0",9600,timeout=1)
time.sleep(1) # wait for serial to be ready
ser.write(query)
time.sleep(0.1)
reply=ser.readline()
print(reply.strip())
ser.close()
```

Python is rather lightweight, and we have to import extra functionality such as serial communication by executing

```
sudo apt-get install python-serial
```

t the command prompt and then load the new functionality with the import statement.In the above script we import support for handling the serial line and basic time-handling. The latter we need to implement delays in the script. In the second line we define a variable query that contains the query string we send to the Arduino. Note that we explicitely add the carriage return character. Then we open the serial port ser on /dev/ttyACM0 with a baud rate of 9600 and a timeout of 1 second, which prevents unsatisfied read attempts from blocking the program. We wait for one second to allow the operating system to finish opening the serial port, and then submit the query string with the ser.write command. Note that we use the method

write on the serial device ser. Then we wait for 0.1 seconds and read characters up to the CR-LF character with the ser.readline() function into the variable reply. We display the reply with the print command after stripping off leading and trailing white-space characters, such as normal spaces or CR-LF characters. Finally, we close the serial line.

Data Storage:

Flat file

The simplest database is certainly a file containing a time stamp and one or more measurement values, possibly even in human-readable form. This is called a flatfile database, and we create one by using the following Python script:

```
# logger with time, V Ziemann,
161228import serial, time, sys, atexit
def cleanup():
ser.close()
atexit.register(cleanup)
query="A0?\n"
ser=serial.Serial("/dev/ttyACM0",9600,timeout=1)
time.sleep(1) # wait for serial to be ready
while 1:
ser.write(query)
time.sleep(0.1)
reply=ser.readline()
print int(time.time()), reply[3:].strip()
sys.stdout.flush()
time.sleep(1)
```

and store it in a file called ask repeat.py. We recognize the same organization as before, when we discussed Python scripts with importing functionality, registering the cleanup() function, opening the serial line, and repeatedly sending the query string and receiving the reply. The only difference is that we print the Unix time, which is the number of seconds since January 1, 1970, also called the epoch, before the measurement value.

LITE OS:

LiteOS is a lightweight and open-source operating system designed for the Internet of Things (IoT) and other resource-constrained devices. Developed by Huawei, LiteOS is designed to support a wide range of IoT applications, including smart homes, smart cities, industrial automation, and wearable devices.

It is specifically designed to be low-power, efficient, and easy to use.
Key features of LiteOS include:

Efficiency: LiteOS is designed to be lightweight and efficient, making it suitable for devices with limited processing power, memory, and battery life. It has a small code footprint and minimal resource requirements, allowing it to run on low-cost, low-power microcontrollers.

Connectivity: LiteOS provides built-in support for a variety of communication protocols, including Wi-Fi, Bluetooth, Zigbee, and NB-IoT, allowing devices to connect to a wide range of networks and services.

Security: LiteOS includes security features such as secure boot, secure communication, and secure storage, to protect devices and data from unauthorized access.

Modularity: LiteOS is designed to be modular and scalable, allowing developers to select and customize the components they need for their specific application.

Ecosystem: LiteOS is supported by a growing ecosystem of hardware and software partners, making it easier for developers to find compatible hardware, tools, and libraries.

Developer-Friendly: LiteOS is designed to be easy to use and developer-friendly, with a comprehensive set of development tools, documentation, and tutorials.

LiteOS is used in a wide range of IoT devices and applications, including smart home appliances, industrial sensors, wearable devices, and more. It is suitable for both commercial and open-source projects, and its lightweight design makes it particularly well-suited for battery-powered and low-cost devices.
LiteOS is an open-source project, and its source code is available on GitHub. Developers can contribute to the project,

report issues, and request new features through the LiteOS GitHub repository.



Real Time Operating System:

Open WSN:

The OpenWSN8 project is an open-source implementation of a fully standards-based protocol stack for IoT networks. It was based on the new IEEE802.15.4e time- slotted channel-hopping standard. IEEE802.15.4e, coupled with IoT standards such as 6LoWPAN, RPL and CoAP, enables ultra-low-power and highly reliable mesh networks that are fully integrated into the Internet.

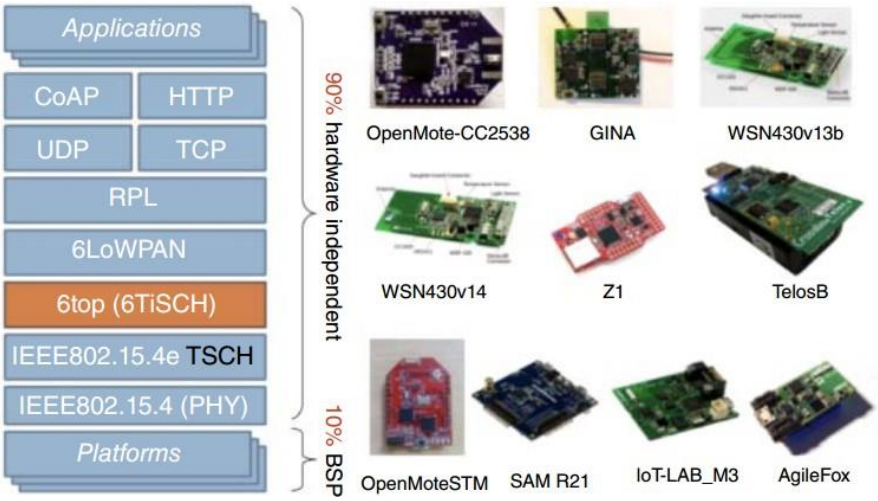


Figure 1: OpenWSN protocol stack, highlighting hardware-independent modules and supported hardware platforms.

OpenWSN has been ported to numerous commercial available platforms from older 16-bit micro-controllers to state-of-the-art 32-bit Cortex-M architectures. The OpenWSN project offers a free and open-source implementation of a protocol stack and the surrounding debugging and integration tools, thereby contributing to the

overall goal of promoting the use of low-power wireless mesh networks. Figure 7.8 shows the OpenWSN protocol layers and software libraries, which are hardware independent, and a set of supported hardware platforms, on which it can be installed and used.

FreeRTOS

FreeRTOS10 is a real-time operating system kernel for embedded devices designed to be small and simple. It been ported to 35 micro-controllers and it is distributed under the GPL with an optional exception. The exception permits users? proprietary code to remain closed source while maintaining the kernel itself as open source, thereby facilitating the use of FreeRTOS in proprietary applications.

In order to make the code readable, easy to port, and maintainable, it is written mostly in C, (but some assembly functions have been included to support architecture-specific scheduler routines). It provides methods for multiple threads or tasks, mutexes, semaphores and software timers.

TI-RTOS

TI-RTOS11 is a real-time operating system that enables faster development by eliminating the need for developers to write and maintain system software suchas schedulers, protocol stacks, power-management frameworks and drivers. It is provided with full C source code and requires no up-front or runtime license fees. TI-RTOS scales from a low-footprint, real-time preemptive multitasking kernel to a complete RTOS with additional middleware components including a power manager, TCP/IP and USB stacks, a FAT file system, and device drivers, allowing developers to focus on differentiating their applications.

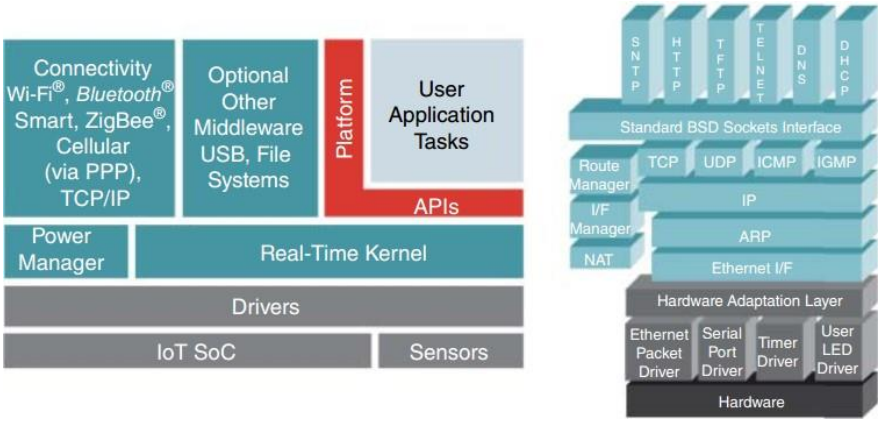


Figure 2: Schematic overview of the TI-RTOS operating system with main modules and software components.

Figure shows the main software components of the TI-RTOS operating system. In particular, it is based on a core layer with a real-time kernel, connectivity support and power management. On top of that, a set of platform APIs allow the developer to build custom applications. The OS provides a large set of ready-to-use libraries based on TCP/UDP/IP networking, standard BSD socket interface and main application layer protocols such as HTTP, TFTP, Telnet, DNS, and DHCP.

RIOT:

RIOT12 is an open-source microkernel operating system for the IoT, licensed as LGPL. It allows C and C++ application programming, and provides both full multi-threading and real-time capabilities (in contrast to other operating systems with similar memory footprints, such as TinyOS or Contiki). RIOT runs on 8-bit (e.g., AVR Atmega), 16-bit (e.g., TI MSP430) and 32-bit hardware (e.g., ARM Cortex). A native port also enables RIOT to run as a Linux or MacOS process, enabling theuse of standard development and debugging tools such as GNU Compiler Collection, GNU Debugger, Valgrind, Wireshark, an so on. RIOT is partly POSIX-compliant and provides multiple network stacks, including IPv6, 6LoWPAN and standard protocols such as RPL, UDP, TCP, and CoAP.

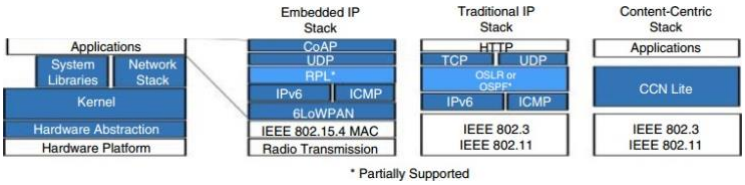


Figure 3: Overview of networking architecture for the RIOT operating system.

RIOT OS is a real-time operating system (RTOS) specifically designed for the Internet of Things (IoT) and other embedded systems. It is an open-source project with a strong focus on low power consumption, scalability, and modularity, making it suitable for a wide range of IoT applications, from small sensor nodes to more complex devices.

RIOT OS supports a large number of microcontrollers and hardware platforms, making it highly flexible and customizable.

Key features of RIOT OS include:

Efficiency: RIOT OS is designed to be highly efficient in terms of memory usage, processing power, and energy consumption, making it suitable for low-power devices.

Scalability: RIOT OS is highly scalable and can be used in a wide range of devices, from small sensor nodes to more complex devices with multiple processors and communication interfaces.

Modularity: RIOT OS is built on a modular architecture, allowing developers to choose and integrate only the components they need for their specific application. This helps reduce code size and complexity and improve overall system efficiency.

Real-time Capabilities: RIOT OS provides real-time capabilities, allowing it to meet strict timing requirements and deadlines, making it suitable for real-time applications such as industrial automation and robotics.

RIOT OS includes security features such as secure boot, secure communication, and support for hardware security features, making it suitable for secure IoT applications.

Compatibility: RIOT OS is compatible with a wide range of hardware platforms, microcontrollers, and communication protocols, making it highly versatile and flexible.

Ecosystem: RIOT OS is supported by a growing ecosystem of developers, contributors, and hardware manufacturers, making it easier to find compatible hardware, tools, and libraries.

RIOT OS is used in a wide range of IoT applications, including smart homes, industrial automation, wearable devices, smart agriculture, and more. It is suitable for both commercial and open-source projects, and its open-source nature makes it highly customizable and adaptable to different requirements.

RIOT OS is an open-source project, and its source code is available on GitHub. Developers can contribute to the project, report issues, and request new features through the RIOT OS GitHub repository.

Contiki OS:

Contiki is an operating system for networked, memory-constrained systems, targeting low-power wireless IoT devices. Its main characteristics are:

It is open source and in continuous development. Even if it is less well documented and less well maintained than commercial operating systems, it allows developers not only to work on custom applications but also to modify core OS functionalities such as the TCP/IP stack and the routing protocol

It provides a full TCP/uIPv6 stack using 6LoWPAN for header compression, and creates LR-WPAN routes with RPL, the IPv6 routing protocol for low- power and lossy networks.

Contiki was created by Adam Dunkels in 2002 and has been further developed by a worldwide team of developers from Texas Instruments, Atmel, Cisco, ENEA, ETH Zurich, Redwire, RWTH Aachen University, Oxford University, SAP, Sensinode, the Swedish Institute of Computer Science, ST Microelectronics, Zolertia, and many others.

Contiki is designed to run on classes of hardware devices that are severely constrained in terms of memory, power, processing power, and communication bandwidth. For example, in terms of memory, despite providing multitasking and a built-in TCP/IP stack, Contiki only needs about 10 kB of RAM and 30 kB of ROM. A typical Contiki system has memory of the order of kilobytes, a power budget of the

order of milliwatts, processing speed measured in megahertz, and communication bandwidth of the order of hundreds of kilobits/second. This class of systems includes various types of embedded systems as well as a number of old 8-bit computers.

A brief description of the core features of Contiki will be provided, highlighting why they are of particular interest for building complex IoT applications.

Networking

Contiki provides three network mechanisms:

the uIP13 TCP/IP stack, which provides IPv4 networking;

the uIPv6 stack, which provides IPv6 networking

the Rime stack, which is a set of custom lightweight networking protocols designed specifically for low-power wireless networks.

The IPv6 stack was contributed by Cisco and was, at the time of release, the smallest IPv6 stack to receive IPv6-ready certification. The IPv6 stack also contains the RPL routing protocol and the 6LoWPAN header compression and adaptation layer.

The Rime stack is an alternative network stack that is intended to be used when the overhead of the IPv4 or IPv6 stacks is prohibitive. The Rime stack provides a set of communication primitives for lowpower wireless systems. The default primitives are single-hop unicast, single-hop broadcast, multi-hop unicast, network flooding, and address-free data collection. The primitives can be used on their own or combined to form more complex protocols and mechanisms.

Low-Power Operation

Many Contiki systems are severely power-constrained. Battery operated wireless sensors may need to provide years of unattended operation, often with no way to

recharge or replace batteries. Contiki provides a set of mechanisms for reducing the power consumption of the system on which it runs. The default mechanism for attaining low-power operation of the radio is called ContikiMAC. With ContikiMAC, nodes can be running in low-power mode and still be able to receive and relay radio messages.

Simulation

The Contiki system includes a network simulator called Cooja . Cooja simulates networks of Contiki nodes. The nodes may belong to one of three classes:

emulated nodes, where the entire hardware of each node is emulated;

Cooja nodes, where the Contiki code for the node is compiled and executed on the simulation host;

Java nodes, where the behavior of the node must be reimplemented as a Java class.

A single Cooja simulation may contain a mixture of nodes from any of the three classes. Emulated nodes can also be used, so as to include on-Contiki nodes in a simulated network.

In Contiki 2.6, platforms with TI MSP430 and Atmel AVR microcontrollers can be emulated. Cooja can be very useful because of its emulative functions, whichhelp developers in testing applications. This speeds up the development process:without a simulator the developer would have to upload and test every new versionof firmware on real hardware. This would be a long process, because most motescan only be flashed through a (slow) serial port.

Programming Model

To run efficiently on memory-constrained systems, the Contiki programming model is based on protothreads. A protothread is a memory-efficient programming abstraction that shares features of both multi-threading and event-driven programming to attain a low memory overhead. The kernel invokes the protothread of a process in response to an internal or external event. Examples of internal events are timers that fire,or messages being posted from other processes. Examples of external events are sensors that trigger, or incoming packets from a radio neighbor.

Protothreads are cooperatively scheduled. This means that a Contiki process must always explicitly yield control back to the kernel at regular intervals. Contiki processes may use a special protothread construct to avoid waiting for events while yielding control to the kernel between each event invocation.

Features:

Contiki supports per-process optional preemptive multi-threading, inter-process com- munication using message-passing events, and an optional GUI subsystem with ei- ther direct graphic support for locally connected terminals or networked virtual display with virtual network computing or over Telnet.

AfullinstallationofContikiincludesthefollowingfeatures:

- multitaskingkernel
- optionalper-applicationpre-emptivemultithreading
- protothreads
- TCP/IPnetworking,includingIPv6
- windowingsystemandGUI
- networkedremotedisplayusingvirtualnetworkcomputing
- webbrowser(claimedtobetheworld?ssmallest)

TinyOS:

TinyOS9 is a free, open-source, BSD-licensed OS designed for lowpower embedded distributed wireless devices used in sensor networks. It has designed to support the intensive concurrent operations required by networked sensors, with minimal hardware requirements. TinyOS was developed by University of California, Berkeley, Intel Research, and Crossbow Technology. It is written in the nesC(Network Embedded Systems C) programming language, which is a version of C optimized to support components and concurrency. It is also component-based, supporting event-driven programming of applications for TinyOS.

TinyOS is an open-source operating system designed for low-power wireless devices, such as sensor networks and internet of things (IoT) devices.
It is developed and maintained by a community of researchers and developers around the world. TinyOS is designed to be highly efficient, flexible, and extensible, allowing developers to create custom applications for resource-constrained devices.
Key features of TinyOS include:

Component-based architecture: TinyOS is built around the concept of components, which are small, reusable units of code that can be combined to create complex applications. This makes it easy to develop and maintain applications, and to reuse code across different projects.
Event-driven programming model: TinyOS is based on an event-driven programming model, where components react to events, such as the receipt of a sensor reading or a timer expiration. This allows applications to be highly responsive and efficient.
Low-power operation: TinyOS is designed to run on devices with limited power resources, such as batteries or energy-harvesting systems. It includes power management features that allow devices to enter low-power states when not in use, extending battery life.
Support for a wide range of hardware platforms: TinyOS supports a wide range of hardware platforms, including popular microcontrollers such as the MSP430 and ARM Cortex-M, as well as radio transceivers such as the CC2420 and CC2538.
A rich set of libraries and tools: TinyOS includes a rich set of libraries and tools for developing applications, including libraries for working with sensors, communication protocols, and power management. It also includes a powerful build system, called nesC, that allows developers to write code in a C-like language and automatically generate efficient, optimized code for their target platform.
TinyOS has been widely used in research and development projects in academia and industry, and has been deployed in a variety of real-world applications, including environmental monitoring, industrial automation, and smart cities.

TinyOS is typically programmed using nesC (network embedded systems C). NesC is a dialect of C that provides abstractions for building networked embedded systems, and which is often used for programming TinyOS applications.

To learn more about TinyOS and its features, you can visit the official website at http://www.tinyos.net. There you can find documentation, tutorials, and sample applications to help you get started with TinyOS programming.
